



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: M. Samanvi

Reg. No.: 192425318

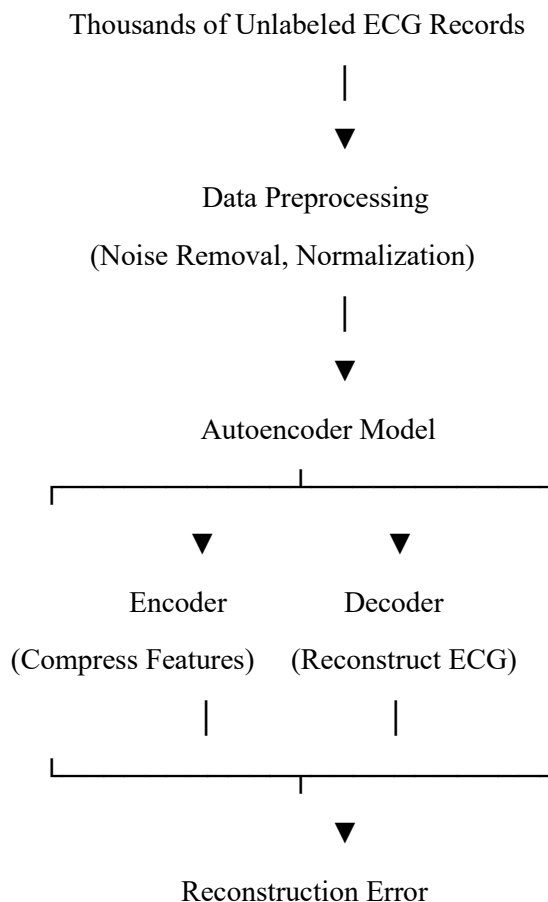
Course Code: MLA0401

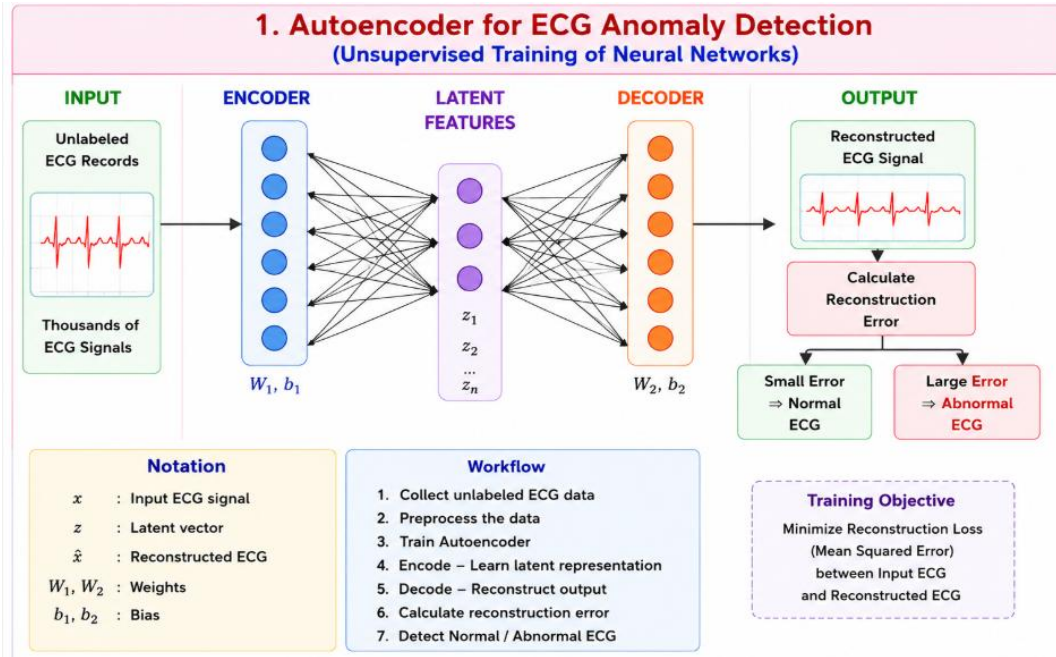
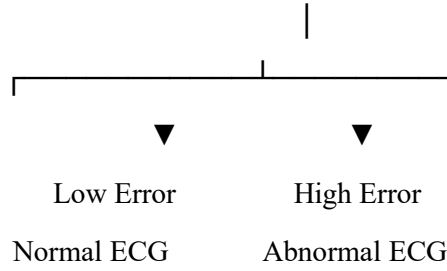
Course Name: Deep Learning

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Workflow:





Dataset:

```

ecg.csv
C:\Users\SAMANVI\Downloads> saveetha cgl > DL ecg.csv

1  F1,F2,F3,F4,F5,F6,F7,F8,F9,F10,F11,F12,F13,F14,F15,F16,F17,F18,F19,F20,F21,F22,F23,F24,F25,F26,F27,F28,F29,F30,F31,F32,F33,F34,F35,F36,F37,F38,F39,F40,F41,F42,F43,F44,F4
2  0.4967141530112327,-0.13826430117118466,0.6476885381006925,1.5230298564080254,-0.2341533747233597,-0.2341369564918055,1.5792128155073915,0.7674347291529088,-0.46947438
3  0.22745993460412942,1.307142754282428,-1.6074832345612275,0.1846338585323042,0.25988279424842353,0.7818228717773104,-1.236950710878082,-1.3204566130842763,0.521941565616
4  0.1135173452124804,0.6621306745210467,1.586016816145352,-1.237815498826849,2.133033746562666,-1.9520877995225019,-0.15178509503558332,0.5883172064845765,0.280991867735
5  5.441933298512231,1.6994175681748682,-1.5141195832512298,0.1575346621725876,-1.4366363053753566,-1.5237897178360063,0.051823655744352104,0.1776377026038618,1.1154612865
6  0.2897748568964129,2.075400798645439,0.8711247034316923,-0.32602353216784113,1.2012139221639448,-0.4080753730215514,-2.038124535177854,-1.008086310917404,-1.870791921025
7  -1.192732995872848,5.809620922024029,-2.7548287742519726,1.6998159819672092,-0.9526745972228968,-0.5455701138234434,-1.2889809680022004,4.703314514545665,-2.13416170678
8  -0.04946370965243295,0.6748194921666038,-1.1227220215622986,0.3824097461840506,0.1664522082130561,0.4924512640081491,0.2891686439078181,2.4553001399108942,-0.63773998425
9  1.6554710197019888,0.1380567644832651,-2.516541801920855,3.720837624528329,-1.1911337934045112,-2.520281627294465,0.8836943326945447,0.3186626490500122,-2.7086515368444
10  1.1055259329579838,1.1870303055063801,0.6387302220291846,-1.1430049127304835,1.633431532301096,-1.146345393018688,0.3026354652390323,-0.7542758500193579,-0.064138346592
11  0.5583269125217224,0.0760053914124115,0.5387559924463692,-0.9206735930793662,0.169360824036078,-1.4137144969049802,-0.11122606134870032,0.9039076414647897,-0.7355299425
12  -1.0469109826819396,0.5366527525727862,1.185704154662997,0.7189533106650667,0.99604768580858164,-0.7567950885694608,-1.4218106667418438,1.5013336519554867,-0.322679838842
13  -0.027514905050636255,1.772251596669744,1.66125921454787,-0.457096255658627184,-0.602212063617805,0.46877425607502377,-0.9983854391882915,0.30179189981946597,0.766080278
14  0.40373009033660945,-0.024195606390916165,-0.9037018607301281,0.324359281721603,-1.1790397911598025,1.1876793884615637,-0.4646172972983931,0.2011596558728393,0.283287871
15  -0.13125696709230472,0.07685188295876821,-0.22485598180807856,-0.6500025808234597,0.16865467196672768,0.44194064956048904,-1.090399077424846,1.41093237974627,-0.09858813
16  -0.05489400061628639,0.28555407485765694,0.5211224283793383,0.645215585950057,0.5556044660034485,0.08958068125524882,-0.1973384203073261,-0.1512732836535117,-0.194908280
17  -0.25372339729589916,0.7468105441317984,-0.5574392914352873,0.21610750565920555,-0.30307658678101446,1.669117825452845,0.14318072656418845,-2.9847035944278355,1.323703534
18  0.19589445311523424,0.5844129804724842,1.9764405845154855,-1.5642416117110514,1.617212574968624,0.10435593752321293,-0.8987840483231282,-1.3303136257389898,-0.1891203907
19  -0.9637589262106109,-0.9571509148950429,0.34378788154988543,-0.04865235157045115,0.032796959451186501,-0.7584953347000605,-0.2340071420576733,-0.9242331442683044,0.89019
20  -0.15932046027187207,-0.32669621318636405,-1.0425776946705763,-1.17223375343664,0.4646986354657417,-0.5505519317084877,0.31635832153614474,-0.8852486861242225,0.1810627
21  1.855503134794263,0.22423474453529177,-0.3462465060608996,-1.611567359102828,-0.2626595203161774,-0.33638694541113623,0.9603047962814704,0.46248362406685406,-1.17936912
22  0.7499002394237105,0.27926783821052595,-0.950910729687232,0.31852100479887446,-0.9916966634403105,-0.6328802748847919,0.18932830729237413,0.29724427338012405,-1.18923246
23  0.24365722370271442,-1.0902084067994298,-0.9853396007209299,-0.969747196346474,0.1105331747394013,-0.626315276183007,0.49406391897551966,-2.854626662904391,-0.85685186
24  0.9411818664199697,0.19189773587303607,1.9483426447898131,1.0251091525817735,-0.653893883255765,-0.02731337985143867,-0.04737070254443595,-1.5821767633461272,-0.94039891
25  -1.19452939766035,0.28580328679473294,-0.8451539584261589,0.5533889830744267,-0.03392522592790888,-1.5947559581633324,0.42069963101847885,-0.577920768327106,0.88114672
26  -0.4536363875916828,1.7262265456356325,-0.8873294588557712,0.7717961772356612,0.35625278870608357,1.261519252535516,-0.6671890847775054,-0.732968899123759,1.096069907477
  
```

Program:

```
# =====  
  
# ECG ANOMALY DETECTION USING AUTOENCODER  
  
# =====  
  
# Install TensorFlow (if needed)  
!pip -q install tensorflow  
  
# -----  
# Import Libraries  
# -----  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from google.colab import files  
from sklearn.preprocessing import StandardScaler  
from tensorflow.keras.layers import Input, Dense  
from tensorflow.keras.models import Model  
  
# -----  
# Upload Dataset  
# -----  
print("Upload ECG5000 CSV Dataset")  
uploaded = files.upload()  
  
import os  
  
csv_files = [f for f in os.listdir('/content') if f.endswith('.csv')]  
  
if len(csv_files)==0:  
    raise Exception("No CSV file found!")
```

```
filename = csv_files[0]

print("Dataset Loaded:", filename)

# -----
# Read Dataset
# -----
df = pd.read_csv(filename)

print("\nDataset Shape:", df.shape)
print(df.head())

# -----
# Features & Labels
# -----
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# -----
# Use only Normal ECG for Training
# -----
X_train = X[y==1]

# Entire dataset for testing
X_test = X
y_test = y

# -----
# Normalize Data
# -----
scaler = StandardScaler()
```

```

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# Build Autoencoder
# -----
input_dim = X_train.shape[1]

input_layer = Input(shape=(input_dim,))

encoded = Dense(64, activation='relu')(input_layer)
encoded = Dense(32, activation='relu')(encoded)
latent = Dense(16, activation='relu')(encoded)

decoded = Dense(32, activation='relu')(latent)
decoded = Dense(64, activation='relu')(decoded)
output = Dense(input_dim, activation='linear')(decoded)

autoencoder = Model(input_layer, output)

autoencoder.compile(optimizer='adam',
                    loss='mse')

autoencoder.summary()

# -----
# Train Model
# -----
history = autoencoder.fit(
    X_train,
    X_train,

```

```

    epochs=50,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)

# =====

# GRAPH 1
# Training Loss
# =====

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'],label="Training Loss")
plt.plot(history.history['val_loss'],label="Validation Loss")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()

# -----
# Reconstruction
# -----

reconstructed = autoencoder.predict(X_test)

mse = np.mean(np.square(X_test-reconstructed),axis=1)

# =====

# GRAPH 2
# Normal ECG
# =====

```

```
normal_index=np.where(y_test==1)[0][0]
```

```
plt.figure(figsize=(12,4))
```

```
plt.plot(X_test[normal_index],  
         label="Original ECG")
```

```
plt.plot(reconstructed[normal_index],  
         label="Reconstructed ECG")
```

```
plt.title("Normal ECG")
```

```
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# GRAPH 3
```

```
# Abnormal ECG
```

```
# =====
```

```
abnormal_index=np.where(y_test!=1)[0][0]
```

```
plt.figure(figsize=(12,4))
```

```
plt.plot(X_test[abnormal_index],  
         label="Original ECG")
```

```
plt.plot(reconstructed[abnormal_index],
```

```

        label="Reconstructed ECG")

plt.title("Abnormal ECG")

plt.xlabel("Time")
plt.ylabel("Amplitude")

plt.legend()
plt.grid()

plt.show()

# =====
# GRAPH 4
# Reconstruction Error Histogram
# =====

plt.figure(figsize=(8,5))

plt.hist(mse[y_test==1],
        bins=40,
        alpha=0.6,
        label="Normal ECG")

plt.hist(mse[y_test!=1],
        bins=40,
        alpha=0.6,
        label="Abnormal ECG")

plt.title("Reconstruction Error Distribution")
plt.xlabel("Reconstruction Error")
plt.ylabel("Count")

```



```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# Threshold
```

```
# =====
```

```
threshold=np.mean(mse[y_test==1])+3*np.std(mse[y_test==1])
```

```
print("\nThreshold:",threshold)
```

```
predictions=(mse>threshold).astype(int)
```

```
actual=(y_test!=1).astype(int)
```

```
accuracy=np.mean(predictions==actual)
```

```
print("\nAccuracy:",round(accuracy*100,2,"%"))
```

Output:

Model: "functional"

...

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 140)	0
dense (Dense)	(None, 64)	9,024
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 32)	544
dense_4 (Dense)	(None, 64)	2,112
dense_5 (Dense)	(None, 140)	9,100

Total params: 23,388 (91.36 KB)

Trainable params: 23,388 (91.36 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/50

13/13 ————— 2s 24ms/step - loss: 1.0108 - val_loss: 0.9988

Epoch 2/50

13/13 ————— 0s 9ms/step - loss: 0.9995 - val_loss: 0.9968

Epoch 3/50

13/13 ————— 0s 10ms/step - loss: 0.9964 - val_loss: 0.9956

Epoch 4/50

13/13 ————— 0s 9ms/step - loss: 0.9929 - val_loss: 0.9943

Epoch 5/50

13/13 ————— 0s 9ms/step - loss: 0.9883 - val_loss: 0.9927

Epoch 6/50

13/13 ————— 0s 9ms/step - loss: 0.9820 - val_loss: 0.9900

Epoch 7/50

13/13 ————— 0s 9ms/step - loss: 0.9742 - val_loss: 0.9866

Epoch 8/50

13/13 ————— 0s 9ms/step - loss: 0.9651 - val_loss: 0.9831

Epoch 9/50

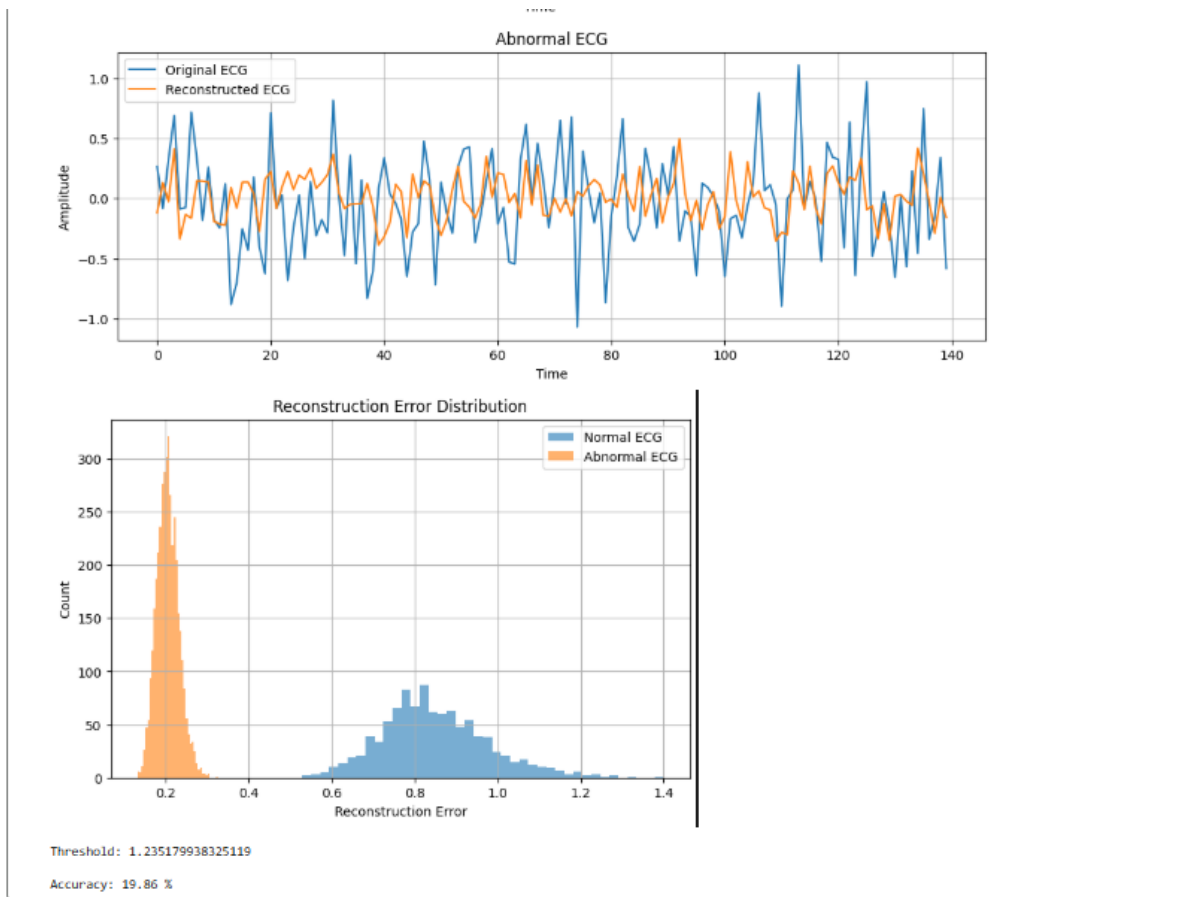
13/13 ————— 0s 9ms/step - loss: 0.9563 - val_loss: 0.9796

Epoch 10/50

13/13 ————— 0s 9ms/step - loss: 0.9481 - val_loss: 0.9772

Epoch 11/50

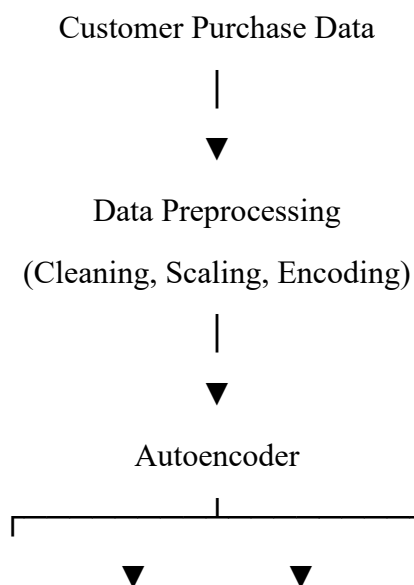
13/13 ————— 0s 9ms/step - loss: 0.9409 - val_loss: 0.9751

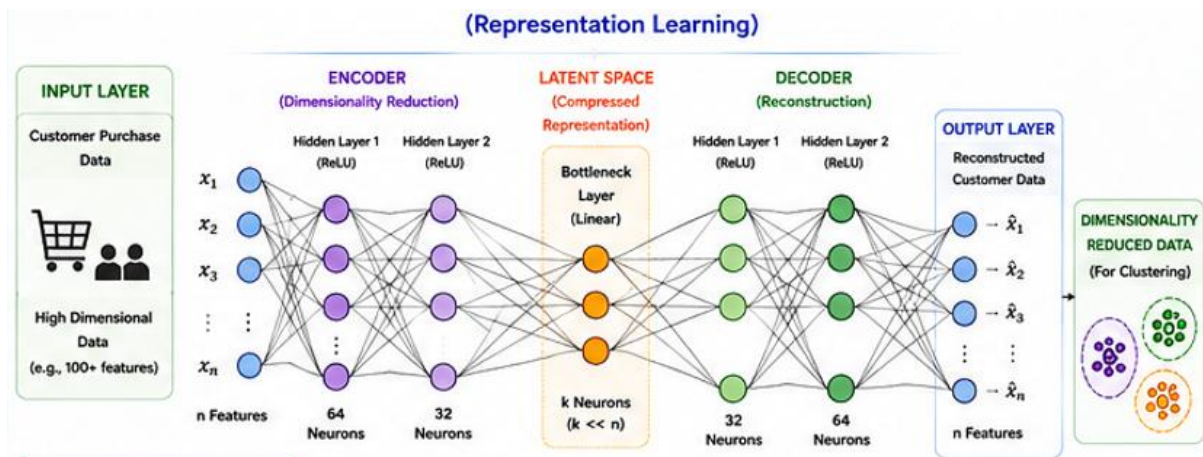
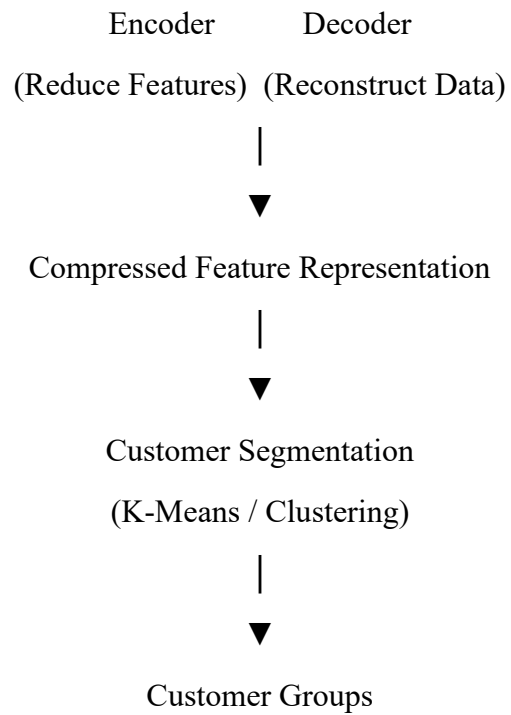


Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem

Workflow:





Datasets:

```
ecg.csv customer_purchase_data.csv X
C: > Users > SAMANVI > Downloads > saveetha dg > DL > customer_purchase_data.csv
1 CustomerID, Age, AnnualIncome, SpendingScore, PurchaseFrequency, AveragePurchase, TotalSpent, OnlineVisits, DiscountUsage, LoyaltyPoints, Segment
2 100001, 56, 16920, 27, 21, 877.72, 6186.71, 50, 7, 5967, Bronze
3 100002, 69, 121121, 40, 11, 853.62, 29245.88, 53, 15, 6428, Bronze
4 100003, 46, 91872, 97, 30, 965.23, 34750.61, 46, 9, 2078, Bronze
5 100004, 32, 96132, 84, 21, 585.57, 44559.87, 52, 1, 2495, Gold
6 100005, 60, 17093, 14, 6, 909.57, 43768.43, 29, 15, 2261, Bronze
7 100006, 25, 27905, 100, 9, 810.43, 48910.99, 5, 3, 1737, Silver
8 100007, 38, 136759, 69, 1, 895.72, 25847.88, 67, 18, 3096, Gold
9 100008, 56, 45981, 87, 12, 21.51, 41622.66, 74, 8, 7927, Platinum
10 100009, 36, 88903, 79, 26, 966.24, 21650.44, 95, 2, 2444, Platinum
11 100010, 40, 85821, 6, 14, 478.79, 21484.54, 75, 10, 336, Platinum
12 100011, 28, 124200, 33, 6, 800.2, 13040.74, 81, 7, 2983, Gold
13 100012, 28, 97289, 55, 1, 135.93, 41002.75, 40, 6, 9159, Gold
14 100013, 41, 131506, 19, 22, 401.98, 49297.85, 72, 0, 6998, Gold
15 100014, 53, 93513, 77, 2, 278.75, 13188.92, 2, 7, 4998, Gold
16 100015, 57, 65516, 29, 24, 298.74, 3278.51, 68, 16, 8863, Platinum
17 100016, 41, 84212, 61, 11, 132.02, 40869.35, 62, 10, 9495, Silver
18 100017, 20, 148498, 70, 3, 807.71, 31201.39, 80, 6, 8187, Silver
19 100018, 39, 145970, 79, 10, 19.15, 48005.09, 58, 8, 9906, Silver
20 100019, 19, 105009, 80, 30, 319.38, 12985.84, 85, 13, 2595, Gold
21 100020, 41, 59816, 24, 6, 182.67, 4015.87, 69, 4, 2970, Gold
22 100021, 61, 19138, 23, 6, 593.15, 24875.89, 58, 17, 3269, Platinum
23 100022, 47, 106176, 10, 20, 261.71, 4730.57, 35, 12, 478, Silver
24 100023, 55, 90945, 54, 21, 266.41, 13458.61, 14, 4, 843, Silver
25 100024, 19, 19017, 41, 2, 493.45, 6735.9, 15, 7, 1625, Platinum
26 100025, 38, 25451, 49, 26, 656.16, 11594.54, 6, 14, 1880, Bronze
27 100026, 50, 108863, 17, 26, 130.09, 5754.16, 16, 3, 8696, Bronze
28 100027, 29, 25278, 56, 24, 183.76, 47718.35, 87, 0, 8301, Gold
29 100028, 39, 58764, 45, 10, 254.7, 14288.85, 95, 5, 9597, Bronze
```

Program:

```
# =====
# CUSTOMER SEGMENTATION USING AUTOENCODER
# =====

# Install TensorFlow

!pip -q install tensorflow

# Import Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans
```

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

# =====

# Upload Dataset
# =====

print("Upload customer_purchase_data.csv")

uploaded = files.upload()

# =====

# Read Dataset
# =====

df = pd.read_csv("customer_purchase_data.csv")

print(df.head())

print("\nDataset Shape:", df.shape)

# =====

# Encode Segment
# =====

encoder = LabelEncoder()

df["Segment"] = encoder.fit_transform(df["Segment"])
```

```
# Remove CustomerID
```

```
X = df.drop("CustomerID",axis=1)
```

```
# =====
```

```
# Normalize
```

```
# =====
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# =====
```

```
# Autoencoder
```

```
# =====
```

```
input_dim = X_scaled.shape[1]
```

```
input_layer = Input(shape=(input_dim,))
```

```
encoded = Dense(8,activation="relu")(input_layer)
```

```
encoded = Dense(4,activation="relu")(encoded)
```

```
latent = Dense(2,activation="relu")(encoded)
```

```
decoded = Dense(4,activation="relu")(latent)
```

```
decoded = Dense(8,activation="relu")(decoded)
```

```
output = Dense(input_dim,activation="linear")(decoded)
```

```
autoencoder = Model(input_layer,output)
```

```
encoder_model = Model(input_layer,latent)
```

```
autoencoder.compile(optimizer="adam",  
                    loss="mse")
```

```
autoencoder.summary()
```

```
# =====  
# Train  
# =====
```

```
history = autoencoder.fit(  
    X_scaled,  
    X_scaled,  
    epochs=50,  
    batch_size=32,  
    validation_split=0.2,  
    verbose=1  
)
```

```
# =====  
# Graph 1  
# Training Loss  
# =====
```

```
plt.figure(figsize=(8,5))
```



```
plt.plot(history.history["loss"],label="Training Loss")
```

```
plt.plot(history.history["val_loss"],label="Validation Loss")
```

```
plt.title("Training Loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# Reduced Features
```

```
# =====
```

```
compressed = encoder_model.predict(X_scaled)
```

```
# =====
```

```
# KMeans
```

```
# =====
```

```
kmeans = KMeans(n_clusters=4,  
                 random_state=42)
```

```

clusters = kmeans.fit_predict(compressed)

# =====

# Graph 2
# Customer Segments
# =====

plt.figure(figsize=(8,6))

plt.scatter(
    compressed[:,0],
    compressed[:,1],
    c=clusters,
    cmap="viridis"
)

plt.title("Customer Segmentation using Autoencoder")

plt.xlabel("Latent Feature 1")

plt.ylabel("Latent Feature 2")

plt.colorbar(label="Cluster")

plt.show()

# =====

# Graph 3
# Cluster Distribution
# =====

```

```

cluster_counts = pd.Series(clusters).value_counts().sort_index()
plt.figure(figsize=(7,5))
cluster_counts.plot(kind="bar")
plt.title("Customers in Each Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.grid(axis="y")
plt.show()

# =====

# Graph 4

# Feature Importance

# =====

means = df.drop("CustomerID",axis=1).mean()
plt.figure(figsize=(10,5))
means.plot(kind="bar")
plt.title("Average Customer Features")
plt.ylabel("Average Value")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()
print("\nCustomer Segmentation Completed Successfully!")

```

Output:

customer_purchase_data.csv

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving customer_purchase_data.csv to customer_purchase_data.csv

	CustomerID	Age	AnnualIncome	SpendingScore	PurchaseFrequency	
0	100001	56	16920	27	21	
1	100002	69	121121	40	11	
2	100003	46	91872	97	30	
3	100004	32	96132	84	21	
4	100005	60	17093	14	6	

	AveragePurchase	TotalSpent	OnlineVisits	DiscountUsage	LoyaltyPoints	
0	877.72	6186.71	50	7	5967	
1	853.62	29245.88	53	15	6428	
2	965.23	34750.61	46	9	2078	
3	585.57	44559.87	52	1	2495	
4	909.57	43768.43	29	15	2261	

Segment

0 Bronze

1 Bronze

2 Bronze

3 Gold

4 Bronze

Dataset Shape: (5000, 11)

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 10)	0
dense_6 (Dense)	(None, 8)	88
dense_7 (Dense)	(None, 4)	36
dense_8 (Dense)	(None, 2)	10
dense_9 (Dense)	(None, 4)	12
dense_10 (Dense)	(None, 8)	40
dense_11 (Dense)	(None, 10)	90

Total params: 276 (1.08 KB)

Trainable params: 276 (1.08 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/50

125/125 ————— 3s 5ms/step - loss: 0.9888 - val_loss: 0.9642

Epoch 2/50

125/125 ————— 0s 4ms/step - loss: 0.9478 - val_loss: 0.9005

Epoch 3/50

125/125 ————— 0s 3ms/step - loss: 0.8903 - val_loss: 0.8654

Epoch 4/50

125/125 ————— 0s 4ms/step - loss: 0.8669 - val_loss: 0.8487

Epoch 5/50

125/125 ————— 0s 3ms/step - loss: 0.8517 - val_loss: 0.8360

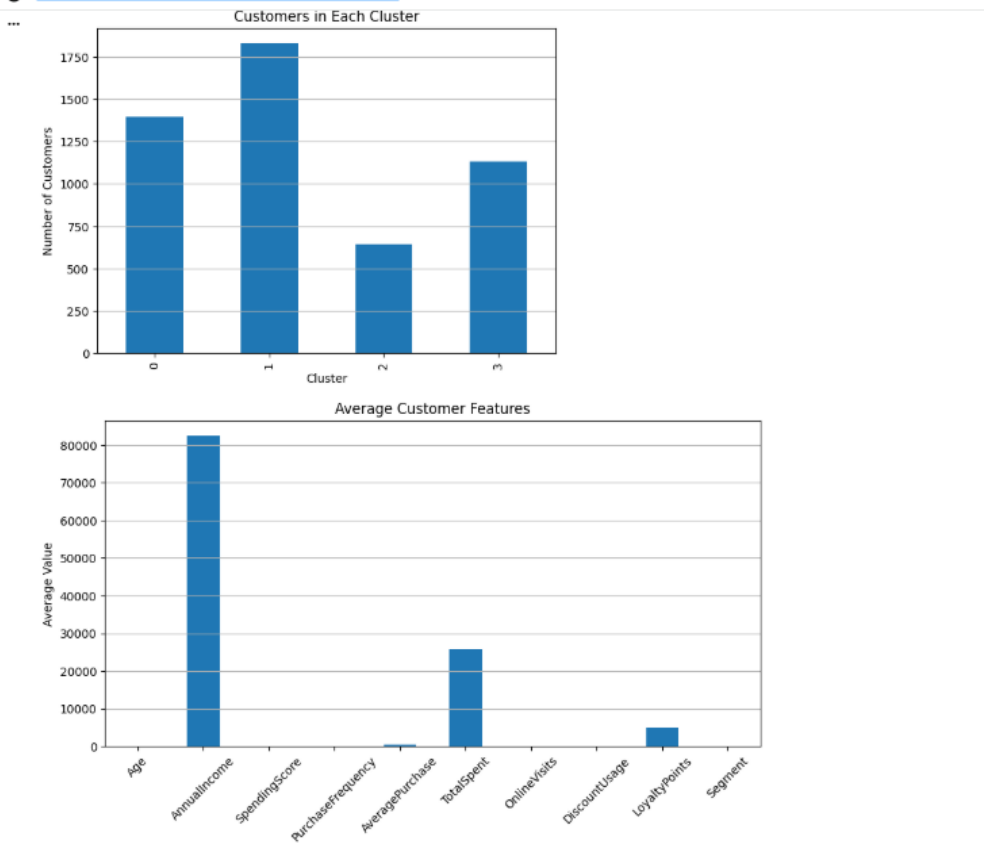
Epoch 6/50

125/125 ————— 0s 4ms/step - loss: 0.8399 - val_loss: 0.8268

Epoch 7/50

125/125 ————— 0s 3ms/step - loss: 0.8208 - val_loss: 0.8100

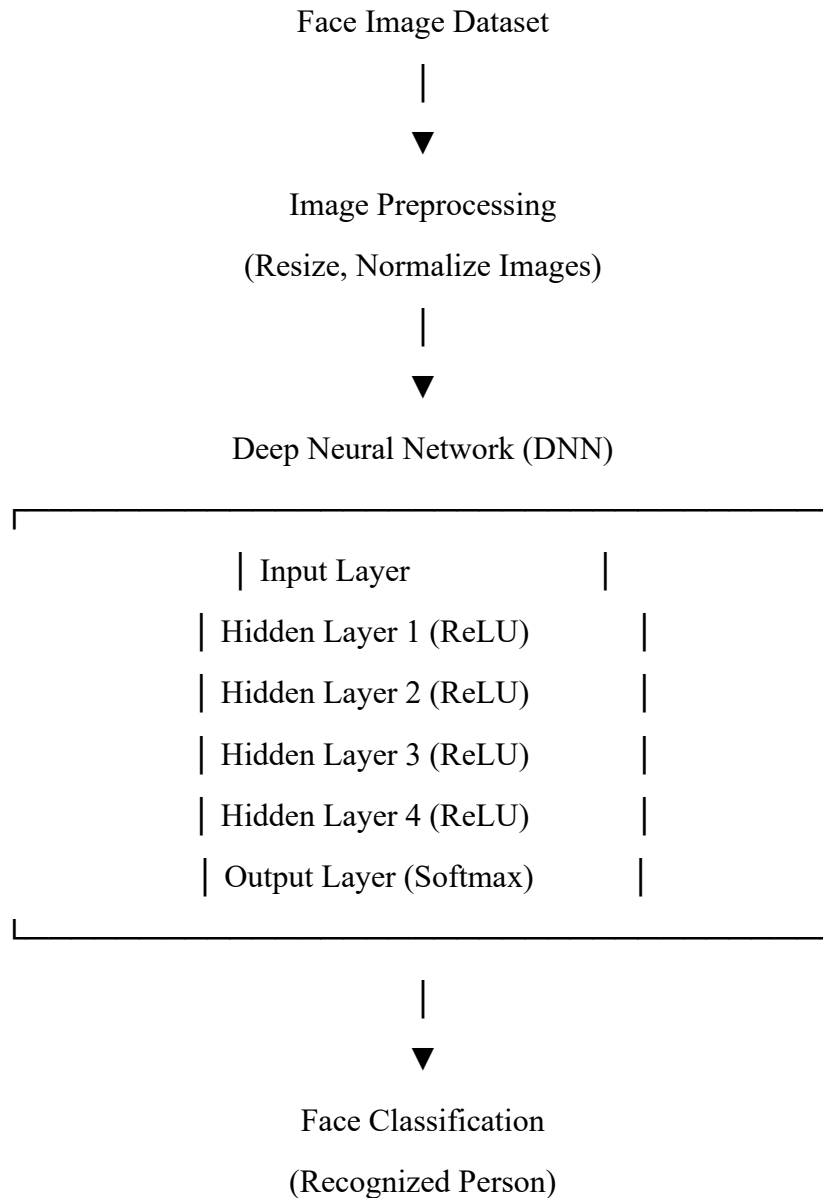
print("\nCustomer Segmentation Completed Successfully!")

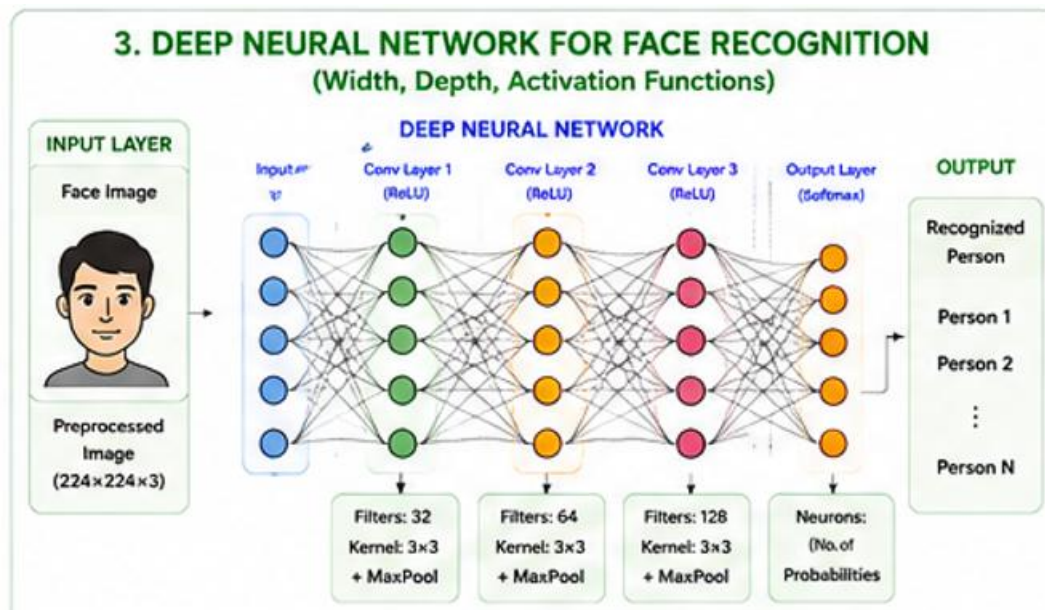


Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions

Workflow:





Datasets:

```

ecg.csv customer_purchase_data.csv face_recognition_features.csv X
C: > Users > SAMANVI > Downloads > saveetha clg > DL > face_recognition_features.csv
1 EyeDistance,NoseWidth,MouthWidth,FaceHeight,JawWidth,ForeheadHeight,LeftEyeSize,RightEyeSize,SmileScore,FaceClass
2 64.48,31.41,51.87,191.75,137.03,61.74,26.66,31.53,0.05,1
3 61.31,33.91,49.99,161.21,130.31,61.42,26.57,29.36,0.2,3
4 65.24,34.05,51.31,176.72,122.7,55.32,30.85,29.35,0.29,9
5 69.62,35.42,54.86,179.58,126.3,62.9,31.73,26.21,0.59,8
6 60.83,29.9,57.11,180.16,132.74,52.55,26.14,31.54,0.81,7
7 60.83,35.78,54.28,181.8,144.55,56.73,26.66,30.75,0.06,9
8 69.9,25.89,52.41,186.1,126.13,50.0,30.29,26.31,0.25,7
9 65.84,32.11,57.99,179.65,136.44,67.79,29.07,27.05,0.38,6
10 59.65,32.54,50.75,176.32,127.38,58.84,29.35,30.51,0.18,5
11 64.71,35.9,56.12,175.92,130.88,70.83,29.79,31.59,0.55,6
12 59.68,34.07,50.85,173.88,142.12,64.01,27.52,25.47,0.48,5
13 59.67,29.78,53.72,187.88,136.11,62.24,28.66,25.99,0.32,2
14 63.21,31.05,51.55,173.53,120.07,57.07,28.5,27.71,0.52,5
15 52.43,34.6,51.78,188.47,125.42,54.77,26.28,23.66,0.36,6
16 53.38,33.76,50.49,180.19,140.05,62.61,28.63,28.04,0.34,5
17 59.19,32.26,57.38,174.79,131.14,59.64,30.48,30.37,0.27,1
18 56.94,32.56,49.07,186.18,120.81,53.3,27.33,25.12,0.54,5
19 63.57,36.09,49.04,172.73,123.45,52.42,28.31,28.25,0.44,1
20 57.46,32.45,60.48,186.38,135.63,55.21,28.41,26.84,0.2,0
21 54.94,33.74,55.91,182.13,137.96,69.1,27.43,31.28,0.01,9
22 69.33,40.51,42.46,174.04,130.14,71.07,29.35,26.86,0.15,9
23 60.87,29.73,49.74,185.29,149.8,61.02,26.64,31.04,0.04,9
24 62.34,28.44,44.75,185.47,136.59,60.74,30.67,26.81,0.43,4
25 54.88,36.35,48.2,191.58,128.05,58.32,29.25,29.54,0.23,3
26 59.28,31.96,39.45,189.66,123.47,55.33,26.13,28.5,0.42,8
27 62.55,33.05,51.45,184.91,122.2,66.88,31.4,27.72,0.33,8
28 56.25,31.66,47.43,168.72,132.68,69.37,28.34,26.42,0.54,8
29 63.88,33.22,52.24,204.49,126.09,55.53,26.55,28.65,0.72,2

```

Program:

```

# =====
# FACIAL RECOGNITION USING DEEP NEURAL NETWORK
# =====

```

```
# Install TensorFlow
!pip -q install tensorflow

# -----
# Import Libraries
# -----

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

# -----
# Upload Dataset
# -----

print("Upload face_recognition_features.csv")

uploaded = files.upload()

# -----
# Read Dataset
# -----
```

```
df = pd.read_csv("face_recognition_features.csv")
```

```
print(df.head())
```

```
print("\nDataset Shape:", df.shape)
```

```
# -----
```

```
# Features and Labels
```

```
# -----
```

```
X = df.iloc[:, :-1].values
```

```
y = df.iloc[:, -1].values
```

```
# -----
```

```
# Split Dataset
```

```
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
# -----
```

```
# Normalize Data
```

```
# -----
```

```
scaler = StandardScaler()
```



```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----
```

```
# Convert Labels
```

```
# -----
```

```
y_train = to_categorical(y_train)
```

```
y_test = to_categorical(y_test)
```

```
# -----
```

```
# Deep Neural Network
```

```
# -----
```

```
model = Sequential()
```

```
model.add(Dense(64,  
                activation='relu',  
                input_shape=(X_train.shape[1],)))
```

```
model.add(Dense(128,  
                activation='relu'))
```

```
model.add(Dense(64,  
                activation='relu'))
```

```
model.add(Dense(32,
```

```
        activation='relu'))

model.add(Dense(10,
                activation='softmax'))

# -----
# Compile Model
# -----

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# -----
# Train Model
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
# =====  
  
# GRAPH 1  
  
# Accuracy  
  
# =====  
  
plt.figure(figsize=(8,5))  
  
plt.plot(history.history['accuracy'],  
         label='Training Accuracy')  
  
plt.plot(history.history['val_accuracy'],  
         label='Validation Accuracy')  
  
plt.title("Model Accuracy")  
  
plt.xlabel("Epoch")  
  
plt.ylabel("Accuracy")  
  
plt.legend()  
  
plt.grid()  
  
plt.show()  
  
# =====  
  
# GRAPH 2  
  
# Loss  
  
# =====
```

```

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'],
         label='Training Loss')

plt.plot(history.history['val_loss'],
         label='Validation Loss')

plt.title("Model Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid()
plt.show()

# =====
# Test Accuracy
# =====

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy:", round(accuracy*100,2), "%")
print("Test Loss:", round(loss,4))

```

Output:

Upload face_recognition_features.csv
No file chosen
Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving face_recognition_features.csv to face_recognition_features.csv

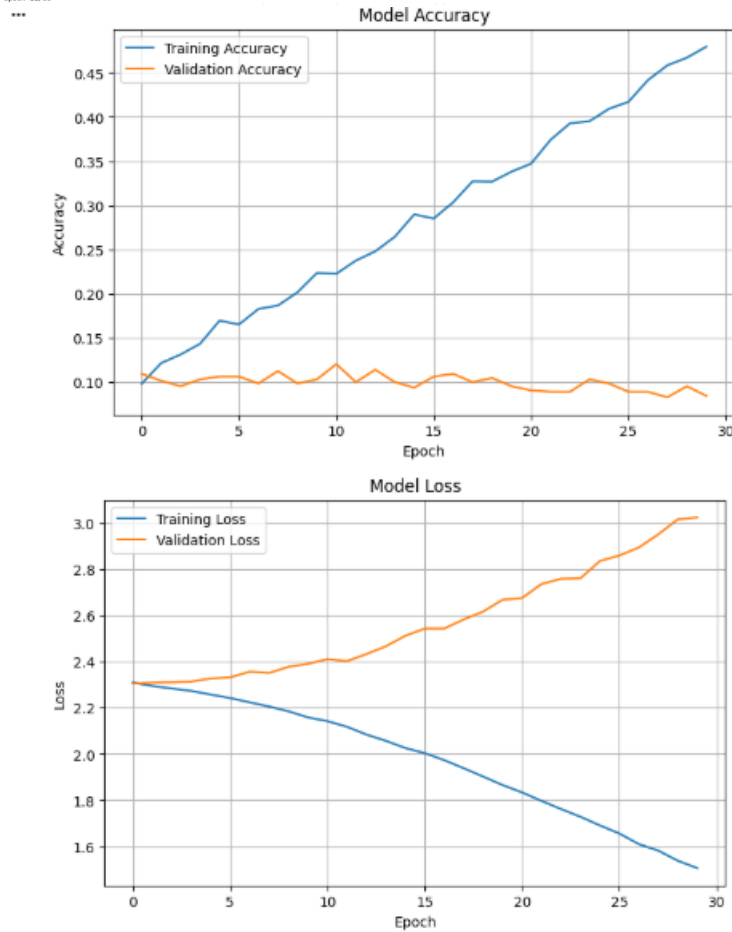
```
EyeDistance NoseWidth MouthWidth FaceHeight JawWidth ForeheadHeight \
0 64.48 31.43 51.87 191.75 117.63 61.74
1 61.31 33.91 49.99 161.21 138.31 61.42
2 65.24 34.05 51.31 176.72 122.78 55.32
3 69.62 35.42 54.86 179.58 126.38 62.88
4 68.83 29.90 57.11 188.16 132.74 52.55
```

```
LeftEyeSize RightEyeSize SmileScore FaceClass
0 26.66 31.53 0.85 1
1 26.57 29.36 0.28 3
2 30.85 29.35 0.29 9
3 31.73 26.21 0.59 8
4 26.14 31.54 0.81 7
```

Dataset Shape: (4000, 16)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:186: UserWarning: Do not pass an 'input_shape' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 64)	640
dense_13 (Dense)	(None, 128)	8,128
dense_14 (Dense)	(None, 64)	8,256
dense_15 (Dense)	(None, 32)	2,080
dense_16 (Dense)	(None, 10)	330

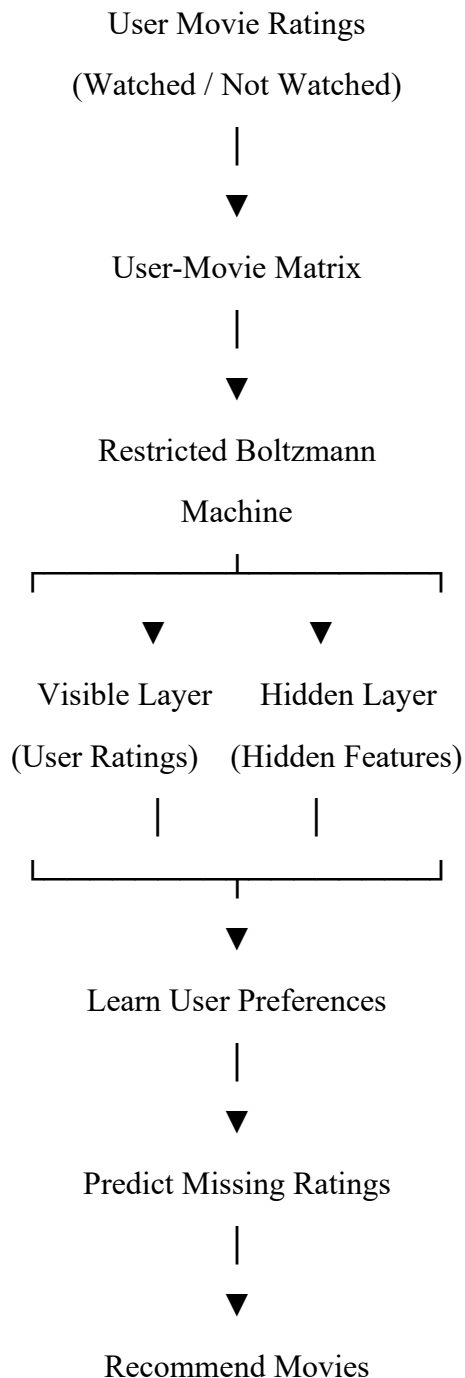
Total params: 19,626 (76.66 KB)
Trainable params: 19,626 (76.66 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/30 11s 58ms/step - accuracy: 0.0980 - loss: 2.3180 - val_accuracy: 0.1094 - val_loss: 2.3059
Epoch 2/30 3s 27ms/step - accuracy: 0.1215 - loss: 2.2952 - val_accuracy: 0.1016 - val_loss: 2.3891
Epoch 3/30 1s 15ms/step - accuracy: 0.1312 - loss: 2.2842 - val_accuracy: 0.0953 - val_loss: 2.3112
Epoch 4/30 2s 19ms/step - accuracy: 0.1434 - loss: 2.2737 - val_accuracy: 0.1031 - val_loss: 2.3143
Epoch 5/30 2s 11ms/step - accuracy: 0.1695 - loss: 2.2581 - val_accuracy: 0.1063 - val_loss: 2.3276
Epoch 6/30 1s 8ms/step - accuracy: 0.1652 - loss: 2.2428 - val_accuracy: 0.1063 - val_loss: 2.3323
Epoch 7/30 0s 4ms/step - accuracy: 0.1828 - loss: 2.2244 - val_accuracy: 0.0984 - val_loss: 2.3570
Epoch 8/30 0s 4ms/step - accuracy: 0.1867 - loss: 2.2062 - val_accuracy: 0.1125 - val_loss: 2.3516
Epoch 9/30 0s 4ms/step - accuracy: 0.2016 - loss: 2.1854 - val_accuracy: 0.0984 - val_loss: 2.3783
Epoch 10/30 0s 4ms/step - accuracy: 0.2234 - loss: 2.1598 - val_accuracy: 0.1031 - val_loss: 2.3911
Epoch 11/30



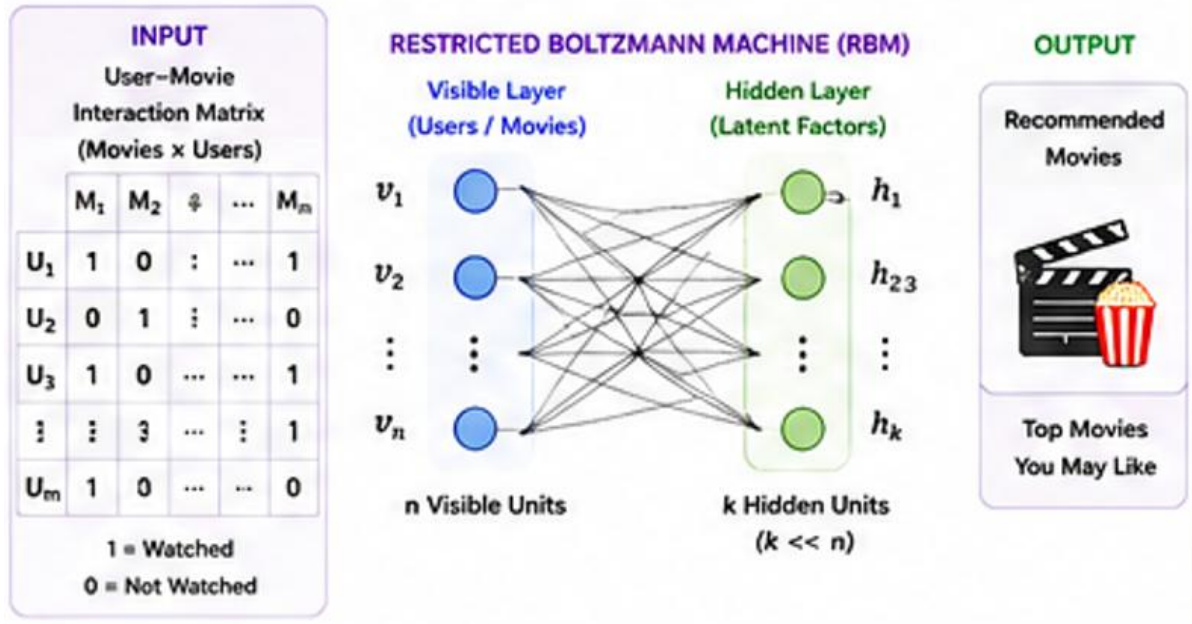
Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

Workflow:



4. RBM FOR MOVIE RECOMMENDATION (Unsupervised Training of Neural Networks)



Datasets:

```

ecg.csv customer_purchase_data.csv face_recognition_features.csv movie_recommendation_data.csv X
C:\Users\SAMANVI>Downloads>saveetha clg>DL> movie_recommendation_data.csv
1 UserID,Movie_1,Movie_2,Movie_3,Movie_4,Movie_5,Movie_6,Movie_7,Movie_8,Movie_9,Movie_10,Movie_11,Movie_12,Movie_13,Movie_14,Movie_15,Movie_16,Movie_17,Movie_18,Movie_19,
2 1,3,5,2,0,4,3,5,0,0,2,0,5,2,3,2,5,0,3,3,0
3 2,4,4,5,4,3,0,1,2,4,0,1,1,2,4,2,3,1,2,2,5
4 3,2,5,3,1,2,3,4,2,2,1,3,0,2,2,5,0,5,0,5,4
5 4,4,5,4,4,2,3,3,1,0,1,2,3,3,0,0,0,2,2,3
6 5,4,3,4,3,3,3,4,4,5,2,3,1,2,5,5,5,2,5,1
7 6,1,5,4,5,4,0,4,0,2,5,1,5,4,1,2,0,0,3,3,3
8 7,2,2,1,1,0,4,1,0,4,5,4,4,3,1,1,2,5,1,2
9 8,2,5,3,3,0,4,4,3,2,2,5,5,2,3,5,0,4,0,2,2
10 9,2,5,1,5,4,1,2,4,0,3,1,5,4,5,5,4,0,4,5,0
11 10,4,5,5,1,5,1,3,0,4,3,4,1,5,1,0,4,3,4,2,2
12 11,3,0,1,2,3,4,5,2,5,2,2,0,4,1,2,3,2,5,1,1
13 12,2,2,2,0,0,4,3,0,4,0,0,0,0,0,2,1,3,4,0,0
14 13,5,2,5,0,2,5,2,2,3,1,1,5,5,4,4,4,2,0,5,1
15 14,4,4,4,3,1,3,4,4,3,0,0,3,5,3,0,5,3,1,0,5
16 15,1,3,0,5,5,4,1,2,3,2,2,0,4,3,4,2,3,0,3,1
17 16,3,5,2,0,2,2,4,3,4,2,1,5,3,1,4,2,5,3,4,2
18 17,5,1,2,5,3,1,4,5,1,1,4,0,2,1,4,2,0,2,2,5
19 18,5,3,3,5,0,4,2,2,4,4,3,3,1,4,3,5,0,2,5,2
20 19,1,3,1,3,2,3,3,5,4,4,4,1,1,4,0,2,3,2,1
21 20,3,2,2,3,0,2,4,5,2,1,4,5,0,2,5,5,4,3,3,2
22 21,4,3,2,3,4,2,3,4,4,4,3,2,1,2,1,0,5,2,3,2
23 22,0,5,4,0,1,5,0,4,4,3,3,2,1,5,5,2,0,5,2,2
24 23,3,5,4,3,4,5,5,0,0,4,5,5,2,2,5,3,0,1,5,3
25 24,1,0,5,0,4,2,3,3,0,4,3,1,2,2,2,3,5,1,1,5
26 25,5,0,3,3,5,1,3,0,4,1,4,3,4,3,0,1,1,1,5
27 26,4,5,3,4,1,3,5,1,4,3,0,1,0,3,4,2,5,3,4,5
28 27,3,2,0,5,0,2,1,1,1,4,5,2,1,2,2,5,0,2,2,1
29 28,0,5,5,3,0,3,4,5,2,5,5,3,3,5,0,3,4,5,2,4
  
```

Program:

```

# =====
# MOVIE RECOMMENDATION USING RBM
# =====
  
```

!pip -q install tensorflow

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.neural_network import BernoulliRBM

# -----
# Upload Dataset
# -----

print("Upload movie_recommendation_data.csv")

uploaded = files.upload()

# -----
# Read Dataset
# -----

df = pd.read_csv("movie_recommendation_data.csv")

print(df.head())

print("\nDataset Shape:", df.shape)

# -----
# Features
# -----
```



```
X = df.drop("UserID", axis=1).values
```

```
# Normalize ratings (0-5 → 0-1)
```

```
X = X / 5.0
```

```
# -----
```

```
# Train-Test Split
```

```
# -----
```

```
X_train, X_test = train_test_split(
```

```
    X,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
# -----
```

```
# Build RBM
```

```
# -----
```

```
rbm = BernoulliRBM(
```

```
    n_components=10,
```

```
    learning_rate=0.01,
```

```
    batch_size=20,
```

```
    n_iter=30,
```

```
    random_state=42
```

```
)
```

```
rbm.fit(X_train)
```

```

# -----
# Hidden Representation
# -----

hidden = rbm.transform(X_test)

# -----
# Reconstruction
# -----

reconstructed = rbm.gibbs(X_test)

# -----
# Reconstruction Error
# -----

error = np.mean((X_test - reconstructed) ** 2, axis=1)

# =====
# GRAPH 1
# Reconstruction Error
# =====

plt.figure(figsize=(8,5))

plt.hist(error,
         bins=25)

plt.title("Reconstruction Error Distribution")

```

```

plt.xlabel("Error")
plt.ylabel("Users")
plt.grid()
plt.show()
# =====

# GRAPH 2
# Hidden Features
# =====

plt.figure(figsize=(8,6))
plt.scatter(
    hidden[:,0],
    hidden[:,1],
    alpha=0.7
)
plt.title("Hidden User Preferences")
plt.xlabel("Hidden Feature 1")
plt.ylabel("Hidden Feature 2")
plt.grid()
plt.show()
# =====

# GRAPH 3
# Average Rating per Movie
# =====

movie_avg = df.drop("UserID", axis=1).mean()
plt.figure(figsize=(12,5))
movie_avg.plot(kind="bar")
plt.title("Average Movie Ratings")

plt.xlabel("Movies")

```

```

plt.ylabel("Average Rating")
plt.grid(axis="y")
plt.show()

# =====

# GRAPH 4

# Recommended Ratings

# =====

plt.figure(figsize=(10,4))
plt.plot(X_test[0],
         marker='o',
         label="Original Ratings")
plt.plot(reconstructed[0],
         marker='s',
         label="Predicted Ratings")
plt.title("Original vs Predicted Ratings")
plt.xlabel("Movies")
plt.ylabel("Normalized Rating")

plt.legend()

plt.grid()

plt.show()

print("\nMovie Recommendation Model Completed Successfully!")

```

Output:

Upload movie_recommendation_data.csv

Choose File

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving movie_recommendation_data.csv to movie_recommendation_data.csv

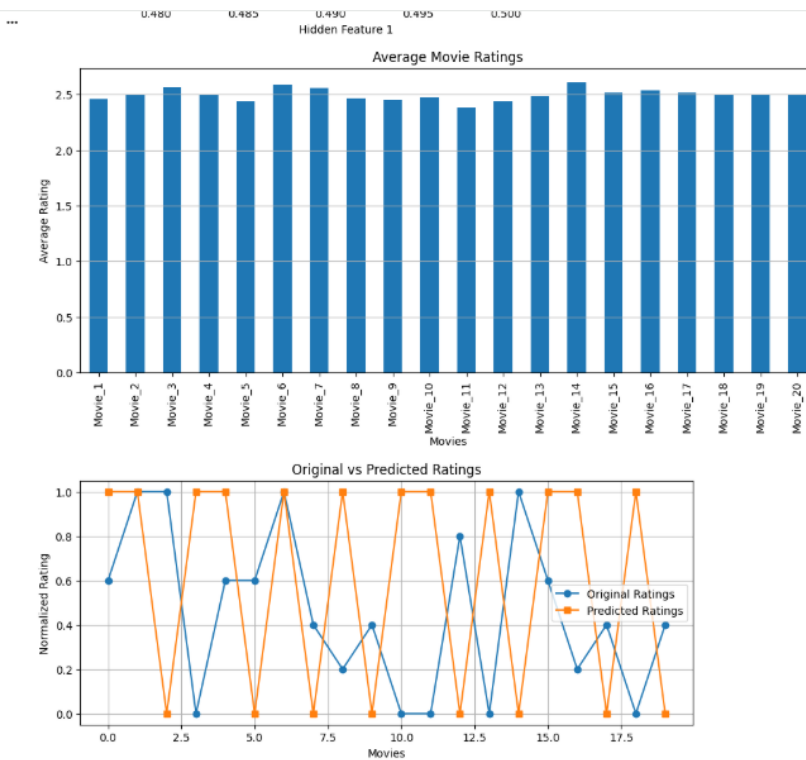
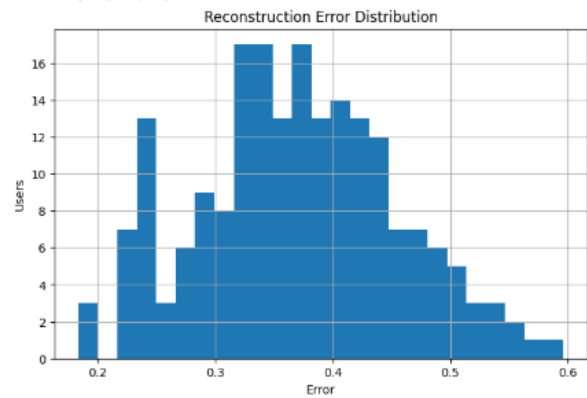
```
UserID Movie_1 Movie_2 Movie_3 Movie_4 Movie_5 Movie_6 Movie_7 \
0 1 3 5 2 0 4 3 5
1 2 4 4 5 4 3 0 1
2 3 2 5 3 1 2 3 4
3 4 4 5 4 4 2 3 3
4 5 4 3 4 3 3 3 3
```

```
Movie_8 Movie_9 ... Movie_11 Movie_12 Movie_13 Movie_14 Movie_15 \
0 0 0 ... 0 5 2 3 2
1 2 4 ... 1 1 2 4 2
2 2 2 ... 3 0 2 2 5
3 1 1 ... 1 2 3 3 6
4 4 4 ... 2 3 1 2 5
```

```
Movie_16 Movie_17 Movie_18 Movie_19 Movie_20
0 5 0 3 3 0
1 3 1 2 2 5
2 0 5 0 5 4
3 0 0 2 2 3
4 5 5 2 5 1
```

[5 rows x 21 columns]

Dataset Shape: (1000, 21)

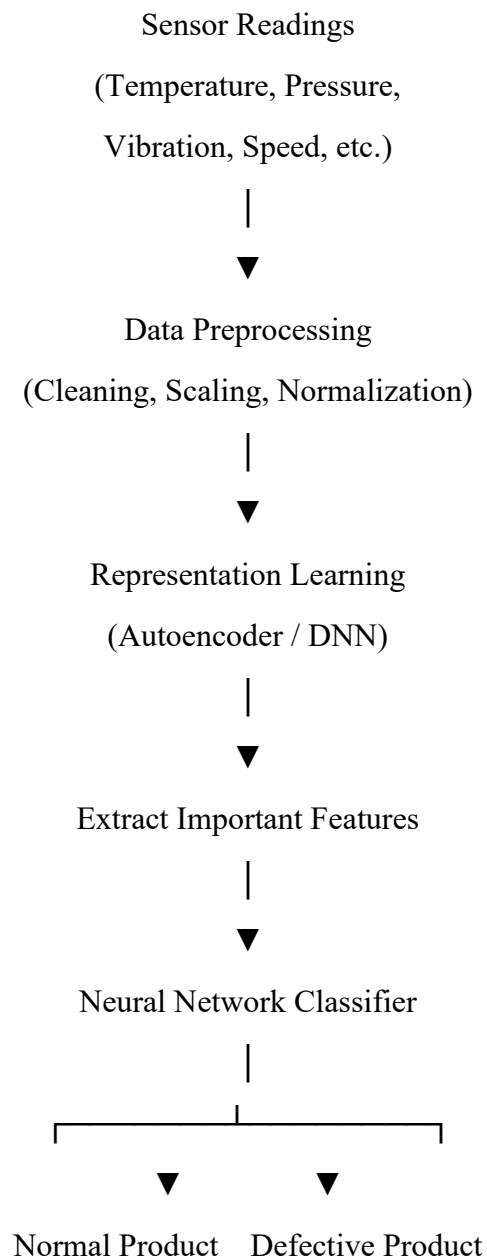


Movie Recommendation Model Completed Successfully!

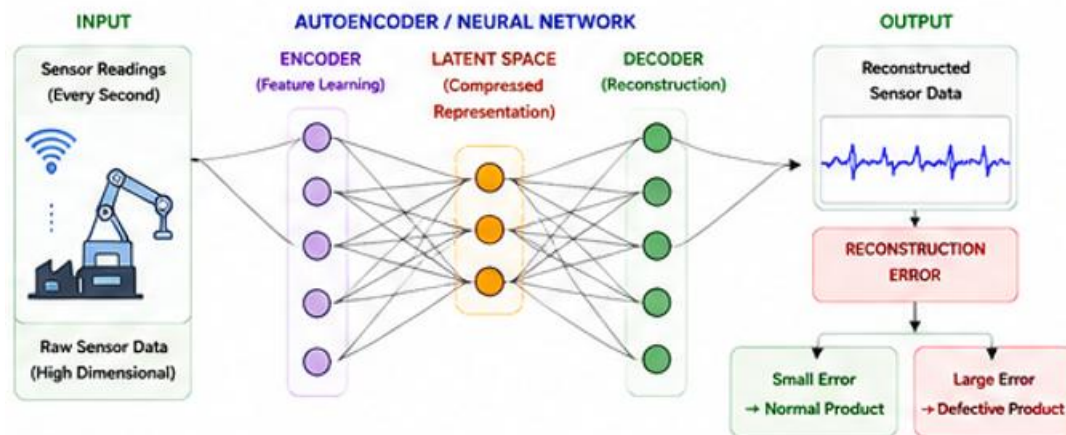
Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks

Workflow:



5. DEEP LEARNING FOR DEFECTIVE PRODUCT DETECTION (Representation Learning + Neural Networks)



Datasets:

```

ecg.csv customer_purchase_data.csv face_recognition_features.csv movie_recommendation_data.csv manufacturing_sensor_data.csv X
C:\Users\SAMANVI>Downloads>saveetha clg>DL>manufacturing_sensor_data.csv
1 Temperature,Pressure,Vibration,Humidity,Voltage,Current,MotorSpeed,Torque,SoundLevel,Defective
2 73.97,28.73,3.32,53.57,222.79,10.26,2915,58.58,59.75,1
3 68.89,28.64,3.69,54.67,222.27,10.02,2521,56.17,66.82,1
4 75.18,24.61,3.4,55.64,212.51,9.35,2983,52.56,54.2,0
5 82.18,29.01,4.11,64.47,224.64,10.0,2899,45.33,61.13,0
6 68.13,32.2,5.2,47.53,208.08,10.74,1963,43.51,58.61,0
7 68.13,26.18,3.23,46.54,214.77,9.81,2495,72.49,58.86,0
8 82.63,33.15,5.0,67.37,204.01,10.63,1966,44.18,67.7,0
9 76.14,31.46,3.22,50.37,232.47,8.74,2687,68.58,60.41,0
10 66.24,27.8,3.15,42.37,218.15,9.09,2331,48.24,55.43,1
11 74.34,29.58,4.82,50.15,237.34,8.9,1458,61.32,64.44,0
12 66.29,34.79,4.92,51.7,226.42,9.15,1513,58.72,64.08,1
13 66.27,32.2,4.85,70.55,223.58,11.93,2526,60.32,54.99,1
14 71.94,30.03,2.68,64.25,215.31,11.09,2067,60.86,54.04,1
15 54.69,29.29,3.53,66.48,211.63,8.38,1040,42.11,62.05,1
16 56.2,30.23,4.82,63.2,224.18,10.66,1837,49.59,59.84,1
17 65.5,28.46,4.04,64.46,219.43,9.41,1262,44.06,57.65,0
18 61.9,23.69,2.93,48.12,209.29,8.98,2317,56.51,65.28,0
19 72.51,36.13,4.46,53.01,207.87,7.77,1684,44.41,57.04,1
20 62.74,30.47,3.29,45.1,212.33,11.46,2032,39.15,62.11,0
21 58.7,31.19,5.79,52.9,234.57,7.95,2076,53.1,65.87,0
22 81.73,29.83,5.54,61.84,237.71,9.92,2819,37.33,58.3,0
23 68.19,27.79,4.6,56.35,221.64,8.58,2022,57.07,50.85,1
24 70.54,29.44,5.36,68.11,221.19,11.16,1127,44.94,57.98,0
25 58.6,25.2,4.06,43.59,217.31,9.5,1530,48.28,60.95,1
26 65.64,36.83,4.77,60.0,212.52,9.8,2987,36.76,65.02,1
27 70.89,26.3,5.48,42.52,231.0,9.85,1643,68.2,57.39,1
28 60.79,26.9,4.25,55.94,234.99,10.45,1524,43.3,68.62,0
29 73.01,28.68,3.74,62.4,212.85,10.02,2755,52.09,53.51,1
30 65.18,31.86,3.7,51.85,210.34,9.04,2861,40.67,66.83,0
  
```

Program:

```

# =====
# DEFECTIVE PRODUCT DETECTION USING
# DEEP NEURAL NETWORK
# =====
  
```

Install TensorFlow

!pip -q install tensorflow

```
# -----  
# Import Libraries  
# -----  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
from google.colab import files  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense  
  
# -----  
# Upload Dataset  
# -----  
  
print("Upload manufacturing_sensor_data.csv")  
  
uploaded = files.upload()  
  
# -----  
# Read Dataset  
# -----  
  
df = pd.read_csv("manufacturing_sensor_data.csv")  
  
print(df.head())
```



```
print("\nDataset Shape:",df.shape)
```

```
# -----
```

```
# Features and Labels
```

```
# -----
```

```
X = df.drop("Defective",axis=1)
```

```
y = df["Defective"]
```

```
# -----
```

```
# Split Dataset
```

```
# -----
```

```
X_train,X_test,y_train,y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

```
# -----
```

```
# Normalize Data
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

# -----
# Build Deep Neural Network
# -----

model = Sequential()

model.add(Dense(
    64,
    activation="relu",
    input_shape=(X_train.shape[1],)
))

model.add(Dense(
    32,
    activation="relu"
))

model.add(Dense(
    16,
    activation="relu"
))

model.add(Dense(
    8,
    activation="relu"
))

model.add(Dense(
```

```
    1,
    activation="sigmoid"
))

# -----
# Compile Model
# -----

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# -----
# Train Model
# -----

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# =====
```

```

# GRAPH 1
# Accuracy
# =====

plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"],
         label="Training Accuracy")

plt.plot(history.history["val_accuracy"],
         label="Validation Accuracy")

plt.title("Model Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.legend()

plt.grid()

plt.show()

# =====

# GRAPH 2
# Loss
# =====

plt.figure(figsize=(8,5))

```

```
plt.plot(history.history["loss"],  
         label="Training Loss")
```

```
plt.plot(history.history["val_loss"],  
         label="Validation Loss")
```

```
plt.title("Model Loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# Predict Defective Products
```

```
# =====
```

```
predictions = model.predict(X_test)
```

```
predictions = (predictions > 0.5).astype(int)
```

```
# =====
```

```
# GRAPH 3
```

```
# Actual vs Predicted
```

```
# =====
```

```
plt.figure(figsize=(10,5))
```

```
plt.plot(y_test.values[:50],  
         marker='o',  
         label="Actual")
```

```
plt.plot(predictions[:50],  
         marker='x',  
         label="Predicted")
```

```
plt.title("Actual vs Predicted Defective Products")
```

```
plt.xlabel("Sample")
```

```
plt.ylabel("Defective (0/1)")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
# =====
```

```
# GRAPH 4
```

```
# Defective vs Normal Products
```

```
# =====
```

```
counts = df["Defective"].value_counts()
```

```
plt.figure(figsize=(6,5))
```

```
counts.plot(  
    kind="bar",  
    color=["green","red"]  
)
```

```

plt.title("Normal vs Defective Products")

plt.xticks(
    [0,1],
    ["Normal","Defective"],
    rotation=0
)

plt.ylabel("Count")

plt.grid(axis="y")

plt.show()

# =====
# Test Accuracy
# =====

loss,accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nTest Accuracy :",round(accuracy*100,2,"%"))

print("Test Loss :",round(loss,4))

Output:

```

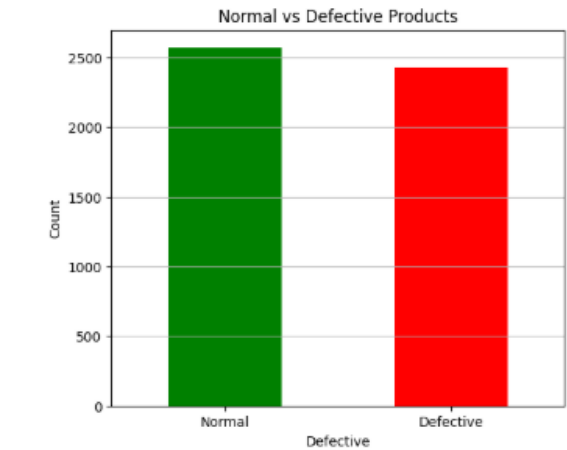
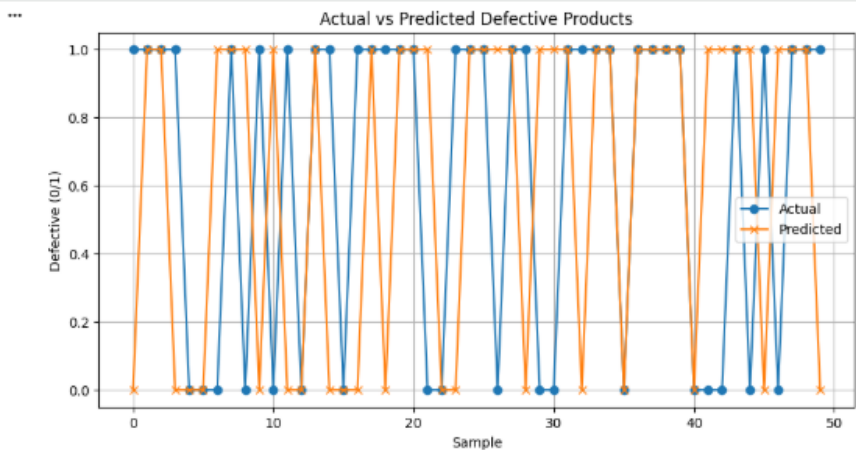
```
... Upload manufacturing_sensor_data.csv
Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Click Here
Saving manufacturing_sensor_data.csv to manufacturing_sensor_data.csv
Temperature Pressure Vibration Humidity Voltage Current MotorSpeed \
0 73.97 28.71 3.32 53.57 222.79 18.26 2915
1 68.89 28.64 3.69 54.67 222.77 18.82 2511
2 75.18 24.61 3.40 55.64 212.51 9.35 2983
3 82.18 29.81 4.13 64.47 224.64 18.80 2899
4 68.13 32.20 5.20 47.53 208.08 18.74 1963

Torque SoundLevel Defective
0 58.58 59.75 1
1 56.17 66.82 1
2 52.56 54.20 0
3 45.23 62.13 0
4 43.51 58.61 0

Dataset Shape: (5000, 10)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:186: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

Layer (type) Output Shape Param #
-----
dense_17 (Dense) (None, 64) 640
dense_18 (Dense) (None, 32) 2,080
dense_19 (Dense) (None, 16) 528
dense_20 (Dense) (None, 8) 136
dense_21 (Dense) (None, 1) 9

Total params: 3,393 (13.25 KB)
Trainable params: 3,393 (13.25 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/30
100/100 -- 3s 7ms/step - accuracy: 0.4956 - loss: 0.6976 - val_accuracy: 0.5063 - val_loss: 0.6935
Epoch 2/30
100/100 -- 0s 5ms/step - accuracy: 0.5284 - loss: 0.6911 - val_accuracy: 0.5163 - val_loss: 0.6937
Epoch 3/30
100/100 -- 1s 4ms/step - accuracy: 0.5412 - loss: 0.6884 - val_accuracy: 0.5200 - val_loss: 0.6932
Epoch 4/30
100/100 -- 0s 4ms/step - accuracy: 0.5556 - loss: 0.6848 - val_accuracy: 0.5088 - val_loss: 0.6972
Epoch 5/30
100/100 -- 0s 4ms/step - accuracy: 0.5566 - loss: 0.6816 - val_accuracy: 0.4762 - val_loss: 0.7014
Epoch 6/30
100/100 -- 0s 4ms/step - accuracy: 0.5728 - loss: 0.6786 - val_accuracy: 0.4875 - val_loss: 0.7037
Epoch 7/30
100/100 -- 0s 4ms/step - accuracy: 0.5769 - loss: 0.6746 - val_accuracy: 0.4963 - val_loss: 0.7082
Epoch 8/30
100/100 -- 0s 4ms/step - accuracy: 0.5831 - loss: 0.6701 - val_accuracy: 0.4850 - val_loss: 0.7120
Epoch 9/30
100/100 -- 0s 4ms/step - accuracy: 0.5969 - loss: 0.6643 - val_accuracy: 0.5025 - val_loss: 0.7148
Epoch 10/30
100/100 -- 1s 5ms/step - accuracy: 0.5941 - loss: 0.6625 - val_accuracy: 0.4800 - val_loss: 0.7150
Epoch 11/30
```



Test Accuracy : 54.5 %
Test Loss : 0.8177